KAKEIBO APP — PROJECT DOCUMENTS

かわいい家計簿
プロジェクト文書一式

本番 URL: https://kawaii0214.vercel.app | 生成日時: 2026/04/24 23:37

目次

1. 仕様書
2. 設計書
3. 取扱説明書
4. セキュリティチェック報告書

01

仕様書

仕様書

1. 目的

家計の入力から分析、予算調整、生活支援までを一元化し、日々の意思決定を支援する。

2. 想定利用者

- 個人の家計を管理したいユーザー
- 生活費、固定費、貯蓄、投資のバランスを見たいユーザー
- AI に相談しながら家計改善を進めたいユーザー

3. 提供機能

3.1 家計入力

- 収入 / 支出 / 貯蓄 / 投資の登録
- カテゴリ、日付、固定費フラグ、メモ入力
- 最近の入力履歴の参照

3.2 ダッシュボード

- 当月収支
- カテゴリ別集計
- 予算消化率
- 安全度 / 生活レベル
- 固定費見直し候補

3.3 予算管理

- 配分プリセット適用
- 予算超過 / 余剰の確認

- トレードオフ提案
- 引き落とし予約の記録

3.4 AI 機能

- AI 分析
- AI 節約アドバイス
- AI チャット相談
- AI 生活支援
- AI 店案内

3.5 認証

- Google ログイン
- LINE ログイン
- メール系認証
- ログアウト

3.6 その他

- 問い合わせ送信
- 設定変更
- 法務ページ表示
- PDF 出力

4. 画面仕様

4.1 ホーム

- ログイン前導線
- ニックネーム / 初期導線
- 認証開始

4.2 ダッシュボード

- 入力、ボード、カレンダー、目標、AI、年次、基準、シニア、こどもを切替

4.3 設定

- デザイン設定
- アカウント設定

4.4 問い合わせ

- 名前、メール、件名、本文を送信

5. API 仕様

代表 API:

- `/api/transactions`
- `/api/profile`
- `/api/preset`
- `/api/budget-surplus`
- `/api/budget-tradeoff`
- `/api/debit-reservations`
- `/api/ai`
- `/api/ai/chat`
- `/api/nearby-shops`
- `/api/contact`

共通仕様:

- JSON リクエスト
- 認証必須 API は Auth0 セッションを確認
- 更新系 API は same-origin を要求
- 一部 API は rate limit 付き

6. AI 店案内仕様

入力:

- 現在地またはエリア文字列
- 店種
- 自由入力キーワード
- 半径

出力:

- 店舗名
- 住所
- 距離
- 緯度経度

- 地図遷移リンク

検索仕様:

- エリア入力時はサーバーで Google Geocoding API（または Nominatim フォールバック）でジオコーディング
- 店舗検索は Google Places API（New）を優先し、失敗または 0 件の場合は Overpass API（OSM）へ自動フォールバック
- Overpass 障害時は予備エンドポイント（overpass.kumi.systems）へ自動切り替え
- 近い順に 6 件まで返却
- 0 件時はエラーコードと検索座標を診断情報としてレスポンスに含む

エラー仕様:

- エリア不明時は `404`
- ジオコーディング失敗時は `502`
- 不正入力時は `400`

7. セキュリティ仕様

- 認証済みユーザーのみ個人データを更新可能
- same-origin 対象 API は cross-site 送信を拒否
- rate limit 対象 API は短時間の多重送信を制限
- 外部向け webhook は署名検証を必須化

8. 品質基準

リリース前に満たす基準:

- `eslint` エラー 0
- TypeScript エラー 0
- `next build` 成功
- 重大なハードコード秘密情報 0

9. 運用ルール

- 実シークレットは Git に含めない
- `.env.local` と Vercel 環境変数を分離する

- DB migration は明示的に適用する

02

設計書

設計書

1. システム概要

本システムは、家計の記録、分析、予算管理、AI 支援を 1 つの Web アプリで提供する Next.js ベースの家計簿です。フロントエンドと API を同一リポジトリの App Router で管理し、データは Supabase、認証は Auth0、配信は Vercel を利用します。

2. 全体構成

```
Browser
  ├─ App Router UI
  ├─ Dashboard / Input / Settings / Contact
  └─ Fetch to /api/*

Next.js App Router
  ├─ src/app/*                画面
  ├─ src/app/api/*            API Route Handlers
  ├─ src/lib/components/*     UI コンポーネント
  ├─ src/lib/auth/*           認証連携
  ├─ src/lib/server/*         セキュリティ / サーバーユーティ
リティ
  └─ src/lib/supabase/*       DB アクセス

External Services
  ├─ Supabase                 profiles / transactions /
budgets
  ├─ Auth0                    Google / LINE / メール認証
  ├─ OpenAI / Gemini 等       AI 機能
  ├─ Google Maps API          周辺店舗検索 / Geocoding（優
先）
  ├─ Nominatim (OSM)          エリアジオコーディング（フォール
バック）
  ├─ Overpass API (OSM)       周辺店舗検索（フォールバック）
  ├─ Resend                   メール送信
  └─ Vercel                   Hosting / Build / Deploy
```

3. レイヤ設計

3.1 Presentation Layer

- `src/app/`
- `src/lib/components/`

役割:

- 家計入力画面、ダッシュボード、設定、法務ページ、問い合わせページを表示
- API 呼び出し結果を反映
- 一部 UI 状態をローカルストレージに保持

3.2 Application Layer

- `src/app/api/`

役割:

- 認証済みユーザーのリクエストを受けて入力検証を実行
- same-origin、rate limit、認可を通した上で業務処理を実行
- 外部 API と DB の橋渡しを行う

3.3 Domain / Utility Layer

- `src/lib/utils.ts`
- `src/lib/fixed-cost-flags.ts`
- `src/lib/components/*` 内の計算ロジック

役割:

- 家計サマリー、予算差分、固定費判定、生活支援ロジックの計算

3.4 Infrastructure Layer

- `src/lib/supabase/admin.ts`
- `src/lib/auth0.ts`
- `src/lib/server/security.ts`
- `src/lib/server/ssrf.ts`

役割:

- Supabase 管理クライアント
- Auth0 セッション処理
- JSON 読み取り制限、same-origin、rate limit

- SSRF 対策

4. データ設計

主要テーブル:

- `profiles`
- `transactions`
- `budgets`

補助データ:

- 予算差額
- 予算トレードオフ
- 引き落とし予約

基本方針:

- ユーザー単位でデータを分離
- Supabase の RLS を前提にする
- サーバー側では Auth0 セッションから解決した `supabaseUserId` を利用して書き込み対象を固定する

5. 認証設計

認証方式:

- Auth0 セッション認証
- Google OAuth
- LINE Login
- メール系ログイン / 確認フロー

設計ポイント:

- 画面系は Auth0 セッション前提
- API はセッション確認と same-origin を併用
- webhook 系 API は same-origin ではなく署名検証で保護

6. セキュリティ設計

実装済みの主要制御:

- `requireSameOrigin` による CSRF 軽減
- `rateLimit` による濫用抑制

- `readJsonBody` によるサイズ制限つき JSON 読み取り
- `boundedText` による入力長制限
- webhook の JWT / 署名検証
- SSRF 対策ユーティリティ

今回の補強:

- `src/app/api/profile/route.ts` に same-origin と rate limit を追加

7. AI 店案内設計

対象:

- `src/lib/components/NearbyShopGuide.tsx`
- `src/app/api/nearby-shops/route.ts`

処理フロー:

1. 画面で店種、エリア、自由入力キーワードを指定
2. エリア入力時: `/api/nearby-shops` に送信 → Google Geocoding API（失敗時は Nominatim）でジオコーディング
3. 現在地使用時: ブラウザの Geolocation API で座標を取得 → `/api/nearby-shops` に送信
4. サーバーで Google Places API（New）を呼び出し（`GOOGLE_MAPS_API_KEY` 設定時）
5. Google が失敗または 0 件の場合、サーバーから Overpass API（OSM）でフォールバック検索
6. 距離順に最大 6 件へ整形し表示
7. 0 件時はエラーコードと検索座標を診断情報としてレスポンスに付与し UI に表示

設計意図:

- サーバーが認証・レート制限・ジオコーディング・店舗検索をすべて担当
- Google Maps API が有効な場合は優先して使用し、失敗または 0 件時は OSM へ自動フォールバック
- OSM（Nominatim / Overpass）は無料・APIキー不要で常に予備として機能
- Overpass 障害時は予備エンドポイント（overpass.kumi.systems）へ自動切り替え
- 0 件時は Google/OSM のエラーコードと検索座標を診断情報として返し、原因の特定を容易にする

8. デプロイ設計

環境:

- ローカル開発
- Vercel Preview
- Vercel Production

本番 URL: `https://kawaii0214.vercel.app`

デプロイ方針:

- Git push → main ブランチ → Vercel 自動デプロイ
- 本番環境変数は Vercel 管理
- `.env.local` はローカル専用

9. 非機能要件

- TypeScript による型安全
- ESLint による静的検証
- Next.js build 通過をリリース基準にする
- セキュリティ境界を API 側で強制する

10. 今後の改善候補

- API ごとの rate limit 方針を明文化
- 監査ログの追加
- README と docs の多言語整理
- AI 店案内の監視指標追加

03

取扱説明書

取り扱い説明書

1. はじめに

このアプリは、収支の記録、予算管理、AI 相談、生活支援をまとめて行う家計簿です。

2. ログイン方法

利用できるログイン方法:

- Google
- LINE
- メール

ログイン後はホームからダッシュボードへ進みます。

3. 基本的な使い方

3.1 収支を入力する

1. 入力タブを開く
2. 種別を選ぶ
3. 金額、日付、カテゴリを入力する
4. 必要に応じてメモや固定費設定を入れる
5. 保存する

3.2 ダッシュボードを見る

確認できる内容:

- 当月の収支
- 予算との差
- カテゴリごとの使い方

- 固定費見直し候補
- 安全度と生活レベル

3.3 予算を設定する

1. プリセット設定を開く
2. 収入や配分を入力する
3. 固定費、変動費、貯蓄、投資の割合を決める
4. 保存してダッシュボードに反映する

4. AI 機能の使い方

4.1 AI 分析

- 家計の状態を文章で確認したいときに使います

4.2 AI チャット

- 家計について質問したいときに使います

4.3 AI 店案内

1. 店種を選ぶ
2. エリアを入力するか、現在地検索を使う
3. 必要なら自由入力キーワードを入れる
4. 検索結果の一覧と地図を確認する

使いどころ:

- 食料品や日用品の買い物先を探したいとき
- 生活圏の候補店を整理したいとき
- 予算に合わせて買い物先を絞りたいとき

5. 設定

設定画面では次の内容を変更できます。

- デザイン
- アカウント情報
- 一部表示設定

6. 問い合わせ

問い合わせページから次を入力して送信できます。

- 名前
- メールアドレス
- 件名
- 本文

7. よくあるトラブル

ログインできない

- ブラウザの Cookie を有効にする
- Auth0 / Google / LINE の設定値を確認する

現在地で店案内が使えない

- ブラウザのアドレスバー横のアイコンから位置情報を許可する
- 通信状況を確認する
- HTTPS 環境でのみ位置情報が利用可能

エリア検索で店が見つからない

- 駅名や市区町村名に変える
- 自由入力キーワードを追加する
- 時間をおいて再試行する（OpenStreetMap サーバーが混雑している場合あり）

周辺のお店情報を取得できない

- しばらく待ってから再試行する
- 別の店種や小さめのエリアで試す

保存できない

- セッション切れの可能性があります。再ログインしてください

8. 運用上の注意

- 実データを扱う場合は本番環境変数を必ず設定してください
- シークレットは `.env.local` または Vercel 環境変数で管理してください

- SQL migration 未適用だと一部機能が正しく動かない場合があります

04

セキュリティチェック報告書

セキュリティチェック報告書

1. 実施概要

- 対象アプリ: かわいい家計簿 / `kakeibo-app`
- 本番 URL: `https://kawaii0214.vercel.app`
- 実施日: 2026-04-24（最終更新: 2026-04-24）
- 実施範囲: 依存パッケージ監査、ビルド検証、静的シークレット検査、API 防御状況の確認、本番 URL 確認

2. 数値サマリー

- セキュリティ総合評価: `93%`
- `npm audit` 脆弱性件数: `0`
- Critical: `0`
- High: `0`
- Moderate: `0`
- Low: `0`
- Git 管理下のハードコード秘密情報検出件数: `0`
- Git 管理下の `.env` 系ファイル件数: `1`
- 管理対象の `.env` 系ファイル: `.env.example` のみ
- `POST` API ルート数: `17`
- `same-origin` 防御あり: `16 / 17`（`94.1%`）
- `rateLimit` 適用あり: `7 / 17`（`41.2%`）
- ビルド失敗件数: `0`
- 非ブロッキング build warning 件数: `2`

3. 総合評価の算出方法

- 今回の総合評価は、静的検査とビルド検証の結果を 100 点満点で重み付けして算出
- 依存脆弱性: `30 / 30`
- シークレット管理: `25 / 25`
- リクエスト起点保護: `18.8 / 20`
- レート制限: `4.1 / 10`
- 品質ゲート: `15 / 15`
- 合計: `92.9 / 100` → 四捨五入して `93%`

3.1 配点の考え方

- 依存脆弱性は `npm audit` の結果をそのまま反映し、`0 件` のため満点
- シークレット管理は Git 管理下のハードコード秘密情報 `0 件` を満点評価
- リクエスト起点保護は `same-origin` を基本指標とし、`16 / 17` のため `94.1%` を 20 点へ按分
- レート制限は `7 / 17` の適用率 `41.2%` を 10 点へ按分
- 品質ゲートは ESLint、TypeScript、`next build` が通過しているため満点

3.2 解釈

- `93%` は「現時点の静的・構成・ビルド観点では高水準だが、追加改善余地あり」という評価
- 主な改善余地は `rateLimit` の適用拡大と、外部からの動的ペネトレーションテスト未実施の 2 点

4. 実施コマンド

- `npm audit --json`
- `./node_modules/.bin/eslint.cmd .`
- `node ./node_modules/typescript/bin/tsc --noEmit --pretty false`
- `npm run build`
- `git grep -n -I -E "(AIza ... |sk_live_ ... |PRIVATE KEY ... |AKIA ... |ghp_ ... )" -- .`
- `git ls-files .env .env.local .env.example .env.preview.local .vercel`
- API ルート走査:

```
POST_ROUTES=17
SAME_ORIGIN=16
```

```
RATE_LIMITED=7
MISSING_SAME_ORIGIN:
src/app/api/auth/send-email/route.ts
```

5. 結果詳細

4.1 依存パッケージ監査

- `resend` SDK 依存を外し、Resend REST API への直接送信に切り替えた後で `npm audit` を再実行
- その結果、依存関係由来の既知脆弱性は `0` 件
- 現時点での依存数は `582` で、そのうち production 依存は `176`

4.2 シークレット管理

- Git 管理下ファイルに対する静的スキャンでは、典型的な API キーや秘密鍵のハードコードは検出されず
- `.env`、`.env.local`、`.vercel` は Git 管理対象外
- Git 管理下に残っている環境変数ファイルは `.env.example` のみ

4.3 API 防御

- `POST` を持つ API ルートは `17`
- そのうち `16` ルートに `requireSameOrigin()` が入っており、一般的なブラウザ起点の CSRF 対策を実施
- `7` ルートに `rateLimit()` が入り、短時間連打の抑制を実施
- `src/app/api/auth/send-email/route.ts` は `same-origin` の代わりに Supabase hook の JWT / 署名検証で保護

4.4 品質ゲート

- ESLint: `0` errors / `0` warnings
- TypeScript: `0` errors
- `next build`: 成功
- ビルド警告は Auth0 依存からの warning が `2` 件あるが、出荷ブロッカーではない

6. 残課題と所見

- `rateLimit` は全 `POST` API に一律ではないため、将来的には対象拡大の余地あり
- Auth0 の `dpopUtils.js` に起因する build warning は依存側の挙動で、現状は機能阻害なし

- 今回は静的検査とビルド検証が中心であり、外部からの動的ペネトレーションテストまでは未実施
- LINE / Google OAuth の Callback URL は本番 URL に合わせて各開発者コンソールへ登録が必要

7. 総合判定

- 今回の実施範囲では、リポジトリ内の依存脆弱性は `0 件`
- 重大な秘密情報の Git 混入も検出されず
- ビルド、型検査、Lint を通過しており、現時点のコードベースはデプロイ可能な状態
- 総合評価は `93%` で、本番運用に耐える状態だが、今後は rate limit の拡張でさらに引き上げ可能